Sugarcane Leaf Disease Classifier with YOLO26

Industrial AI (IAI-26) · Week 8 Assignment · Ghulam Mustafa · Oct 7, 2026

Summary

I trained a YOLO26 model that looks at a photo of a sugarcane leaf and names its disease. On 877 test photos it never saw during training, it was right 873 times (**99.5%**).

- **Problem:** at a sugar mill, field staff check growers' cane for disease by eye. Experts are few and fields are many, so diagnosis is slow.
- **Data:** a public dataset of 6,748 sugarcane leaf photos in 11 classes (9 diseases, Healthy, Dried), licence CC BY 4.0.
- **Model:** YOLO26 nano classifier (`yolo26n-cls`), fine-tuned for 30 epochs.
- **Application:** a field-visit report tool (a folder of photos becomes a CSV with disease and action) and a small web app for one photo.

The most useful findings were about the **data**, not the model:

1. **The dataset has many duplicate photos.** 362 pairs of identical photos carry two different disease labels. I removed those 614 photos.
2. **A normal random split would have cheated.** 403 of 921 test photos (44%) had a copy in the training set. I used a grouped split instead, so no copy is on both sides.
3. **99.5% is the score on this dataset, not in our fields.** Most photos are cut leaves on a tiled floor, and red rot is not one of the classes. The model must be tested on our own field photos before real use.

1. The workplace problem

A sugar mill earns from the sugar in the cane it buys, so diseased cane in growers' fields is lost money for the mill.

The mill does not grow its own cane. It buys it from hundreds of growers. The mill's cane development staff visit fields, look at leaves and advise growers. Today this is done by eye:

- Experienced staff are few, and the fields are many and far apart.
- Photos are sent on for a second opinion, so the advice is late.
- Some diseases (smut, grassy shoot, viral diseases) spread through **seed cane**. If an infected field is used for the next planting, the disease multiplies.

What I built: a tool that takes a photo of a leaf and returns the disease name, how sure it is, and what to do. When it is not sure, it says *refer to an agronomist* instead of guessing.

The two ways the tool can be wrong:

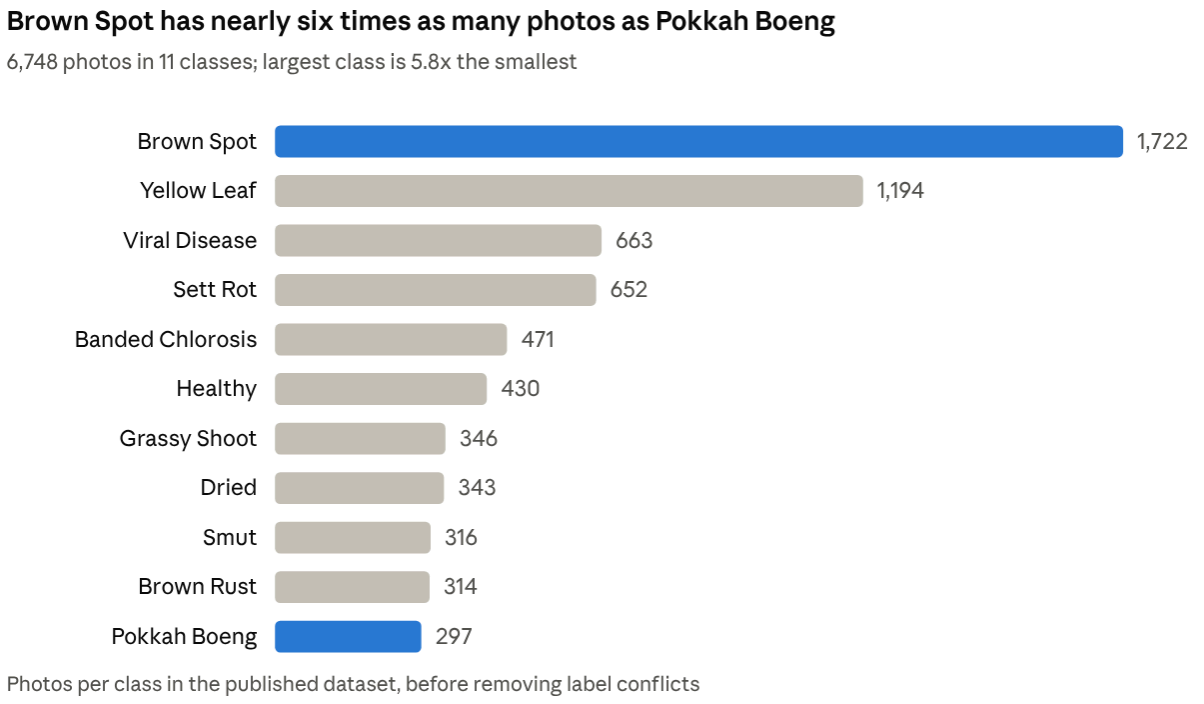
Mistake	What happens	Cost
Missed disease (a diseased leaf called Healthy)	Nobody acts; the disease spreads, maybe into seed cane	High
False alarm (a healthy leaf called diseased)	An officer visits or a grower sprays for nothing	Lower: time and chemicals

Every result in this report counts both.

2. The dataset

I used the public **Sugarcane Leaf Dataset**: 6,748 photos of sugarcane leaves, each with one of 11 labels, free to use under the CC BY 4.0 licence.

- **Source:** Thite, Suryawanshi, Patil and Chumchu, *Data in Brief* (2024), downloaded from Hugging Face (Project-AgML).
- **Classes:** 9 diseases (Banded Chlorosis, Brown Spot, Brown Rust, Grassy Shoot, Pokkah Boeng, Sett Rot, Smut, Viral Disease, Yellow Leaf), plus Healthy and Dried.
- **Split:** none is provided, so I made my own (section 4).



Sugarcane Leaf Dataset, Hugging Face (Project-AgML) · 6,748 photos, 11 classes

The classes are unequal. A model can score high overall just by being good at the big classes, so section 6 checks every class on its own.

Known limits, stated before any result:

- The photos are not from our mill's growers. Most show cut leaves laid on a tiled floor; only some are taken in the field.
- **Red rot**, one of the most damaging cane diseases in our region, is not one of the 11 classes.
- The dataset contains many duplicate photos, some filed under two different diseases (section 4).

3. Choosing the YOLO26 task

I chose **classification**, because each photo shows one leaf with one condition and the dataset gives one label per photo, not boxes.

YOLO26 task	What it answers	Fit for this problem
Classify (`yolo26n-cls.pt`)	One label for the whole photo	**Chosen:** one leaf per photo, labels are per photo
Detect (`yolo26n.pt`)	A box around each object	Needs a box drawn on every lesion; this dataset has none
Segment (`yolo26n-seg.pt`)	The exact outline of each object	Could measure the diseased area in %, but needs outline labels
Pose / OBB	Body keypoints / rotated boxes	Not relevant to leaves

I used the **nano** size, the smallest YOLO26. It is fast enough to run on a phone or a normal office PC, which is where field staff would use it.

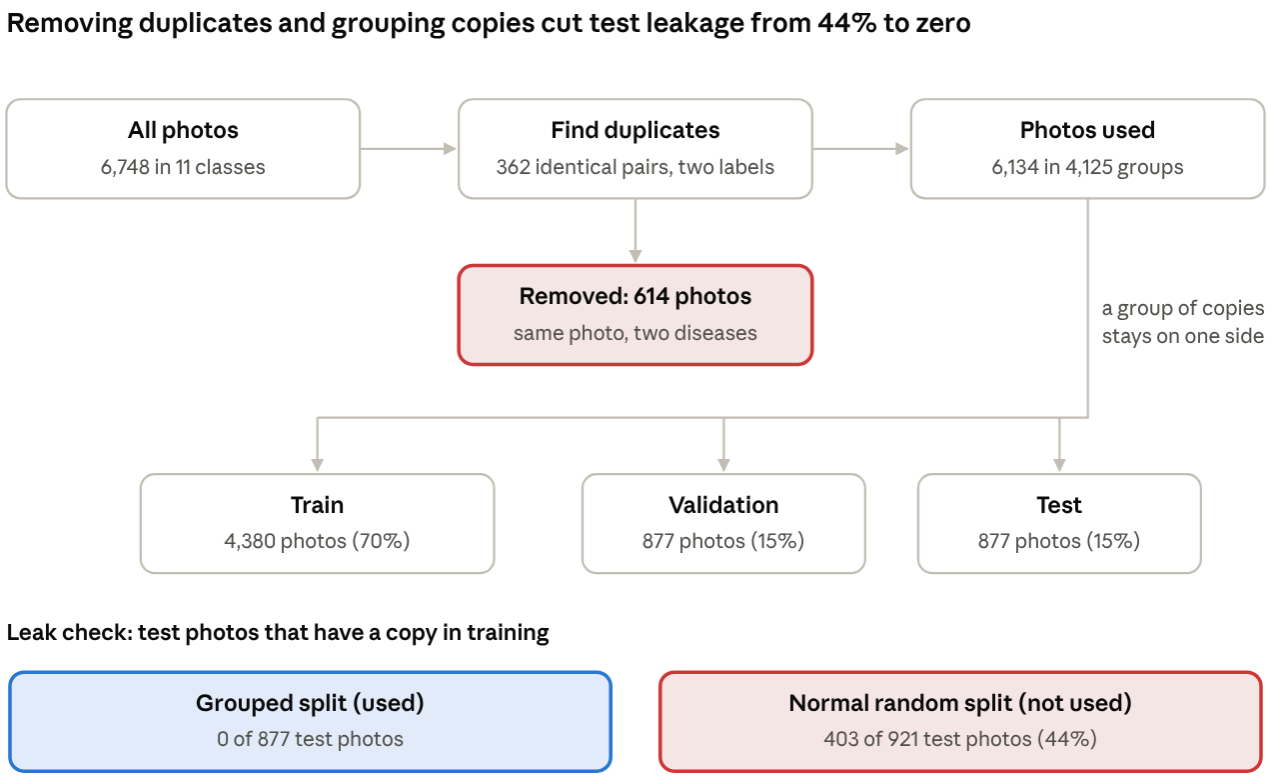
Before training, the model knew nothing about cane. The ImageNet-pretrained model has 1,000 everyday classes and none for sugarcane or any of its diseases. It called a smut leaf a *letter opener* and a brown rust leaf a *rubber eraser*. Fine-tuning teaches it the new words while keeping what it already knows about edges, colours and textures.

4. Checking the data honestly

Before training, I removed 614 photos with contradicting labels and split the rest so that no copy of a photo is in both training and test. A normal random split would have put copies of 44% of the test photos in training.

Why this matters. If the same leaf is in training and in test, the test score measures memory, not diagnosis. The Week 8 class notebook found this same leak in a construction-site dataset.

How I found copies. I shrank every photo to a small 64 x 64 grey thumbnail and measured how similar every pair is (a correlation from 0 to 1). Two photos above 0.95 count as the same leaf. I checked the least similar pairs by eye: they really were the same leaf photographed twice.



data checking and split · 5 steps, leak check compared

- **Copies everywhere:** 5,687 pairs of near-copies; more than half the photos (3,652) have at least one.
- **Label conflicts:** 1,779 pairs are identical (above 0.99), and 362 of them carry two different labels. The most common: Pokkah Boeng / Smut (136 pairs), Banded Chlorosis / Brown Rust (73), Smut / Viral Disease (30). A photo cannot be both, so I removed all 614 such photos.
- **Grouped split:** near-copies are joined into groups (largest group: 46 photos), and a whole group goes to one side. Each class keeps the same share in train, validation and test.

The cost: removing conflicts cut the rarest classes most. Smut fell from 316 to 176 photos and Pokkah Boeng from 297 to 187.

5. Training the model

The model learned fast: validation accuracy was 85.6% after the first epoch and passed 99% by the sixth. Training took 104 minutes on an office PC with no GPU.

Training used the same three lines as the class notebook:

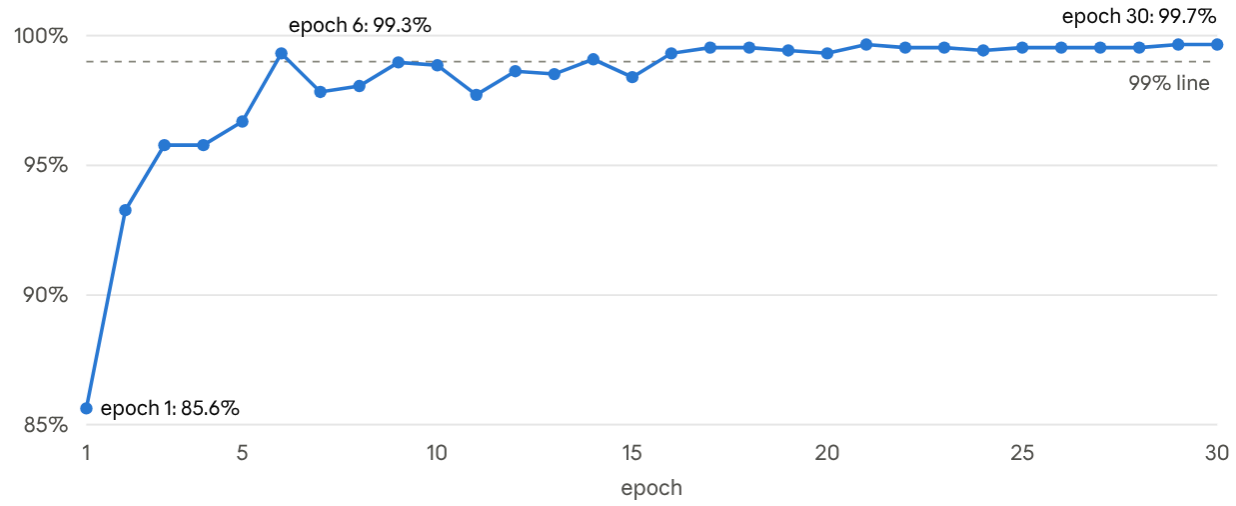
```
model = YOLO("yolo26n-cls.pt")
model.train(data="sugarcane_project/split", epochs=30, imgsz=224)
model.val(split="test")
```

Setting	Value	Why
Starting model	`yolo26n-cls.pt` (ImageNet-pretrained)	Already knows edges, colours and textures
Epochs	30	Enough for the curve below to level off
Image size	224 x 224 pixels	The standard size for this classifier
Batch size	16	Fits a CPU's memory
Random seed	0	Makes the run repeatable
Hardware	CPU only, 10 threads	No GPU on the office PC; Colab's GPU takes about 15 minutes

The classifier learns from folders: each class is a folder of photos, and the folder name is the label. Ultralytics trains on `train/`, picks the best epoch using `val/`, and never looks at `test/`.

Validation accuracy passed 99% by epoch 6 and levelled off near 99.6%

Top-1 accuracy on the 877 validation photos after each training epoch



results.csv from the training run · 30 epochs, 877 validation photos

Training loss and validation loss both fell steadily (from 1.41 to 0.03 and from 0.45 to 0.01), so the model did not overfit: it did not memorise the training photos at the cost of new ones.

6. Results

On the 877 held-out test photos the model was right 873 times: **99.54% top-1 accuracy**, with **no missed disease and no false alarm**.

Measure	Result	Simple meaning
Top-1 accuracy	**99.54%** (873 of 877)	The model's first answer is right
Top-5 accuracy	100%	The right answer is always among its 5 best guesses
Macro-F1	0.9948	The average score over the 11 classes, each counted equally, so rare classes are not hidden
Missed disease	**0 of 771**	No diseased leaf was called Healthy
False alarm	**0 of 60**	No healthy leaf was called diseased

The headline number can hide weak rare classes, so I checked every class on its own. **Recall** is the share of a disease the model found; **precision** is the share of its answers that were right.

Class	Training photos	Test photos	Precision	Recall	F1
Smut	126	25	1.000	0.960	0.980
Pokkah Boeng	134	27	0.964	1.000	0.982
Viral Disease	409	82	1.000	0.988	0.994
Brown Spot	1,214	243	0.988	1.000	0.994
Yellow Leaf	830	165	1.000	0.988	0.994
Dried	228	46	1.000	1.000	1.000
Brown Rust	169	34	1.000	1.000	1.000
Banded Chlorosis	261	52	1.000	1.000	1.000
Sett Rot	465	93	1.000	1.000	1.000
Healthy	298	60	1.000	1.000	1.000
Grassy Shoot	246	50	1.000	1.000	1.000

Sorted worst first.

The two weakest classes are the two rarest. Smut (126 training photos) and Pokkah Boeng (134) score lowest, which matches the lesson from class: rare classes are learned least well. The gap here is small (F1 0.98 against 1.00), but it rests on only 25 and 27 test photos, so one more mistake would change it a lot.

Where the 4 mistakes went: three photos were wrongly called *Brown Spot* (one each of Viral Disease, Smut and Yellow Leaf), and one Yellow Leaf photo was called *Pokkah Boeng*. Brown Spot is the largest class (1,214 training photos), so the model leans towards it when unsure.

7. When the tool should say "I am not sure"

I recommend a **confidence threshold of 0.9**: below it, the photo goes to an agronomist. On the test set this sends 1.3% of photos to a person and lets 3 mistakes through unchecked.

A classifier always gives an answer, even for a blurred photo or a disease it was never taught. A threshold makes it refuse when its confidence is low. This trades two things I can count:

- **Referred:** photos sent to a person (costs agronomist time).
- **Wrong but trusted:** mistakes made with high confidence, which nobody checks (the real risk).

Threshold	Photos referred	Accuracy on the rest	Wrong but trusted
0.0 to 0.7	0%	99.5%	4
0.8	0.1%	99.5%	4
0.9	**1.3%**	**99.7%**	**3**
0.95	1.8%	99.8%	2

The notebook first finds the lowest threshold that meets a 97% accuracy target, which is 0.0 here, and then applies a floor of 0.9, so the tool now uses 0.9. **That is right for this test set but wrong for the field.** Our growers' photos will be harder than these, and red rot is not one of the 11 classes, so the tool must keep a way to say "refer". The floor is a business decision: whoever owns cane-quality risk should confirm it.

8. Looking at the mistakes

Of the 4 wrong answers, by my reading only 1 is clearly the model's fault. The other 3 come from photos that do not show the disease well.

A number alone can mislead, so I looked at each wrong photo and sorted it into one of three causes: **the model is wrong**, **the label is doubtful**, or **the photo cannot decide it**.

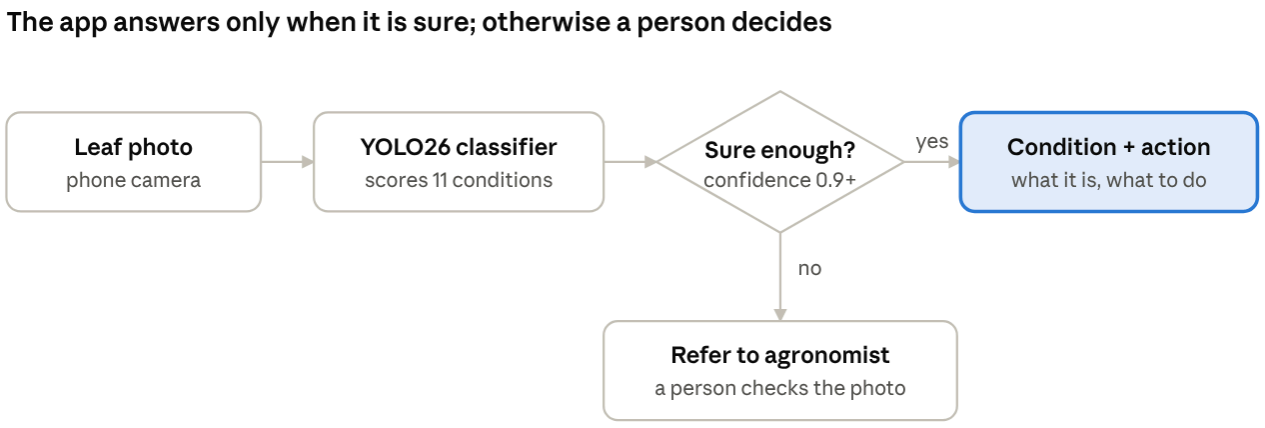
True class	Model said	Confidence	What the photo shows	Cause
Viral Disease	Brown Spot	0.99	A cut leaf on the floor; faint streaks only	Model is wrong
Yellow Leaf	Pokkah Boeng	0.95	A wide field shot of the whole plant; no single leaf in focus	Photo cannot decide it
Smut	Brown Spot	0.92	A purple stalk in the field; the leaf symptoms are not visible	Photo cannot decide it
Yellow Leaf	Brown Spot	0.90	A leaf on the floor, partly cut off at the edge of the photo	Photo cannot decide it

What this means for use: a short photo guide for field staff (one leaf, filling the frame, in daylight) would prevent most of these errors. Only the first kind is fixed by a better model.

Mistakes in the data itself. Section 4 found 362 pairs of identical photos with two different labels, for example the same photo filed as both *Smut* and *Pokkah Boeng*. These are not model mistakes; no model can be right about both. I removed them before training.

9. The application

I built two tools on the trained model: a **field-visit survey report** for a folder of photos, and a **web app** for one photo. Both give the condition, a short explanation and an action, and both refer doubtful photos to a person.



decision flow in both tools · recommended threshold 0.9

Tool 1: field-visit survey report. A field officer photographs leaves across a grower's field and puts the photos in one folder. The tool reads the folder and writes a CSV file with one row per photo, plus a count per condition. In the demo, 24 test photos stood in for one field visit: 23 were answered, all correctly, and 1 was referred because the model was only 0.86 sure. Three rows of the output:

Photo	Condition	Confidence	What it is	Action
00360.jpg	Yellow Leaf	1.00	Virus spread by aphids	Do not use as seed cane; refer
01559.jpg	Banded Chlorosis	0.86	Model not confident enough	UNSURE: refer photo to agronomist
02083.jpg	Sett Rot	1.00	Fungal disease starting in the planted setts	Check germination gaps; refer for seed-treatment advice

Tool 2: web app. A small Gradio page. Upload or take a photo of a leaf and it shows the three most likely conditions with their confidence, the action, or a referral when it is not sure. It runs inside the notebook, on Colab or on this PC.

The advice text is a starting point, not agronomic advice. It is general and short on purpose, and the mill's cane development staff should check and correct it before the tool is used with growers:

Condition	What it is	First action
Banded Chlorosis	Physiological disorder, usually after cold injury	Usually no treatment; record and monitor
Brown Spot	Fungal leaf spot	Record severity; refer if widespread
Brown Rust	Fungal rust	Check the variety's susceptibility; refer if spreading
Dried	Dried or old leaf, not a disease	Photograph a green leaf from the same plant
Grassy Shoot	Phytoplasma, spread by infected seed cane and insects	Rogue out affected clumps; do not use the field as seed cane
Healthy	No disease seen	No action
Pokkah Boeng	Fungal disease	Refer to cane development officer
Sett Rot	Fungal disease starting in the planted setts	Check germination gaps; refer for seed-treatment advice
Smut	Fungal disease; black whip from the shoot top	Rogue out and destroy affected clumps; do not use as seed cane
Viral Disease	Virus symptoms such as mosaic	Do not use as seed cane; refer
Yellow Leaf	Virus spread by aphids	Do not use as seed cane; refer

10. Limitations

The 99.5% is a score on this public dataset. It is not yet a score on our growers' fields, for six reasons:

1. **Not our fields.** The photos come from the dataset authors' surveys. Most are cut leaves on a tiled floor. A phone photo of a leaf still on the plant, in a Pakistani field, looks different.
2. **Red rot is missing.** Red rot is one of the most damaging cane diseases in our region, but it is not one of the 11 classes. The tool will call a red-rot leaf something else, possibly with high confidence.

3. **The split may still be easy.** The grouped split removes near-identical photos. Different photos of the same leaf, taken in the same session on the same floor, can still be on both sides. This can make the test easier than real use.
4. **One photo, one label.** A leaf with two conditions has no correct answer in this setup.
5. **One run.** I trained once, with one random seed. The rare classes have only 25 to 27 test photos each, so small differences between two runs are noise, not progress.
6. **Licence.** Ultralytics YOLO is AGPL-3.0. Coursework use is fine, but using it inside the company, even internally, needs AGPL compliance or an Ultralytics Enterprise licence.

11. Next steps

The most important next step is a real-world test: 100 to 200 leaf photos taken by our own field staff, labelled by an agronomist, scored with the same notebook.

1. **Test on our own fields.** This number, not the 99.5%, decides whether the tool is useful.
2. **Add red rot.** The public Roboflow *sugarcane_redrot* dataset has 701 photos with boxes. A YOLO26 **detector** trained on it, running beside this classifier, would combine two YOLO tasks in one app.
3. **Larger images.** Train at 320 pixels instead of 224; small lesions may be lost at 224.
4. **Measure the leak.** Train once on the naive split and compare its test score with the grouped split, to show how much a leaky split overstates accuracy.
5. **Run on a phone.** Export the model (`model.export(format="tflite")`) and time it on a field officer's phone.
6. **A photo guide** for field staff: one leaf, filling the frame, in daylight.

12. Deliverables and how to run

The whole project is one notebook plus the files it produces, all in the project folder `D:\Week8Code`.

Deliverable	File or folder	What it is
Notebook (code + results)	`sugarcane_leaf_disease_yolo26_classify.ipynb`	Every step, with all outputs and charts saved in it
Trained model	`sugarcane_project/runs/sugarcane_cls/weights/best.pt`	The YOLO26 classifier, 3.2 MB

Deliverable	File or folder	What it is
Results package	`sugarcane_project/sugarcane_results.zip`	Model, training charts, per-class table, threshold table, survey report, summary (11 MB)
Survey report example	`sugarcane_project/field_visit_report.csv`	One row per photo: condition, confidence, what it is, action
Training charts	`sugarcane_project/runs/sugarcane_cls/`	`results.png`, confusion matrices, sample batches
Dataset copy	`sugarcane_project/raw/` and `split/`	All 6,748 photos, and the train / val / test split used
This report	This document	Problem, method, results, limitations

To run it again:

1. **On Google Colab (fastest):** upload the notebook, choose *Runtime > Change runtime type > T4 GPU*, then *Run all*. Allow Google Drive access when asked. Photos and the model are saved to `MyDrive/sugarcane_project`, and the results zip downloads to your PC. About 15 to 25 minutes.
2. **On this PC (no GPU):** open the notebook in VS Code and select the `.venv` kernel. The first full run took about 2 hours: 16 minutes to download, 104 minutes to train. Later runs reuse the saved model and take about 10 minutes, unless you set RETRAIN = True.
3. **Quick check:** set `QUICK_RUN = True` in section 1 to test the pipeline in a few minutes on 40 photos per class. Those numbers are not results.

To use the trained model on any PC (no GPU needed):

```
from ultralytics import YOLO
model = YOLO("best.pt")
result = model.predict("leaf_photo.jpg")[0]
print(result.names[result.probs.top1], float(result.probs.top1conf))
```

References

1. Thite, Suryawanshi, Patil, Chumchu (2024). Sugarcane leaf dataset: a dataset for disease detection and classification for machine learning applications. *Data in Brief* 53, 110268.
2. The dataset on Hugging Face, Project-AgML, CC BY 4.0.
3. Ultralytics YOLO, AGPL-3.0, and its licence terms.
4. YOLO26 documentation and the classification task.
5. Roboflow Universe, sugarcane_redrot, for the red rot next step.
6. Week 8 class notebook, *YOLO: where is it (detection)*: sections 7 to 9 on per-class scores, the leak check and the threshold.